

Contract postconditions and return type deduction

Document number: P1323R1
Date: 2019-01-19
Project: ISO/IEC JTC 1/SC 22/WG 21/C++
Audience subgroup: Core
Revises: P1323R0
Reply-to: Hubert S.K. Tong <hubert.reinterpretcast@gmail.com>

Changelog

Changes from R0

- Pruned discussion points; added summary of the EWG direction instead.
- Added wording to implement the EWG direction for resolving the issue of parsing needing lookup.
- Added wording to implement the EWG direction to enable recursive calls from both preconditions and postconditions.

Issue Description

The interaction between return type deduction and contract postconditions was not raised in the review of the latter feature. Since this introduces the possibility of types and names dependent upon return type deduction, we have a noticeable lack of wording.

Consider:

```
template <typename> struct Tricky { enum { Nested }; };  
template <> struct Tricky<int> { struct Nested { }; };  
auto foo()  
    [[ ensures ret :  
        sizeof(Tricky<decltype(ret)>::Nested) ]] // typename?  
{  
    return 42;  
}
```

It has been brought up that the postcondition could not be analyzed in forward declarations in such a situation.

Somewhat separately, recursive calls appearing within the postcondition are currently ill-formed by N4791 subclause 9.11.4.2 [dcl.attr.contract.cond] paragraph 4, which states that the semantic restrictions of a contract condition is the same as if it appeared as the first *expression-statement* in the body of the function.

Possible solutions

Techniques we have for dealing with the apparent parsing issue include using disambiguators in the style of handling dependent names and delayed parsing. Delayed parsing is already necessary for contract conditions of class members declared within the definition of the class since such contract conditions are complete-class contexts. For templates, the return type cannot be deduced until instantiation; therefore, delayed parsing is not sufficient if an abstract syntax tree is to be formed for the template definition (which various implementations rely on in practice).

The following is an enumeration of possible solutions from Richard Smith:

1. Disallow naming the return value in a postcondition if the function has a deduced return type.
2. Allow such naming, but treat the name of the return value as having a dependent type. This means requiring template and typename disambiguators; behavior would be as if the point of instantiation is wherever the definition of the function occurs.
3. Allow such naming as above for templated functions, and for non-templated functions, allow such naming only for definitions. Delaying the parsing of the postcondition until the return type is known is a possible implementation strategy for the non-templated function case of this option.
4. Allow such naming as above, but apply the dependent-type option for non-templated forward declarations.

Notice that a name introduced for the return value associated with a templated entity declared with a deduced return type is type dependent for all options aside from the first (where introducing such a name is ill-formed).

As to the recursive call prohibition: it could be lifted, but the wording is not presented here. Recursive calls to functions subject to return type deduction are already allowed under 9.1.7.5 [dcl.spec.auto] paragraph 10. Technically, the prohibition could be lifted for preconditions as well as for postconditions.

Summary of the EWG Direction

The EWG discussion settled upon the third option presented. For templated functions with deduced return types, this means that the return value may be named without additional restrictions except that the name of the return value is treated as having a dependent type. For the non-templated functions with deduced return types, this means that naming the return value is prohibited except for definitions. The limitations imposed by the first option, as discussed in the R0 version of this paper, was a factor. The second and fourth options were not chosen because they involve the cost of introducing semantics similar to that of template-dependency in non-template contexts.

EWG was asked about, and gave no objection to, resolving the recursive call prohibition for both preconditions and postconditions.

Proposed resolution

The editing instructions within this document use N4791 as its base wording.

Modify subclause 9.11.4.1 [dcl.attr.contract.syn] paragraph 3:

A contract-attribute-specifier using ensures is a postcondition. It expresses a condition that a function should ensure for the return value and/or the state of objects using a predicate that is intended to hold upon exit from the function. The attribute may be applied to the function type of a function declaration. A

postcondition may introduce an identifier to represent the glvalue result or the prvalue result object of the function. Except for templated functions, the optional identifier shall not be present in non-defining declarations when the declared return type of the function contains a placeholder type (9.1.7.5 [dcl.spec.auto]). [Example:

```
int f(char * c)
    [[ensures res: res > 0 && c != nullptr]];
int g(double * p)
    [[ensures audit res: res != 0 && p != nullptr && *p <= 0.0]];

—end example]
```

Modify subclause 9.11.4.2 [dcl.attr.contract.cond] paragraph 4:

For a contract condition associated with a function whose declared return type contains a placeholder type, the placeholder type is not considered undeduced within the predicate of the contract condition. The predicate of a contract condition otherwise has the same semantic restrictions as if it appeared as the first *expression-statement* in the body of the function it applies to.

Modify subclause 12.7.2.2 [temp.dep.expr] paragraph 3:

An *id-expression* is type-dependent if it contains

- [...]
- an *identifier* associated by name lookup with [...],
- the *identifier* `__func__` ([dcl.fct.def.general]), [...],
- within the conditional-expression of a postcondition ([dcl.attr.contract]) associated with a templated function whose declared return type contains a placeholder type, the *identifier* introduced to represent the result of the function (if any),
- [...]

[...]

Acknowledgements

The author would like to thank Aaron Ballman, Jens Maurer, Richard Smith, and Ville Voutilainen for their feedback on the subject of this paper. As usual, any remaining mistakes are the responsibility of the author.